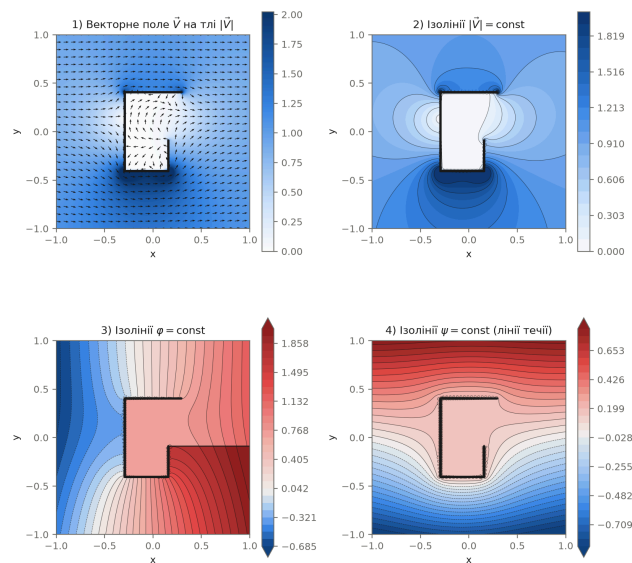


Лабораторна робота №1

Моделювання стаціонарної течії навколо перешкоди заданої форми методом дискретних особливостей

Покроковий гайд

Результат лаб. №1: $M = 60$, $\Gamma_0 = 1$, $\vec{V}_\infty = (1, 0)$, $r_f = 0.01$



Складено за навчальним посібником С.О. Довгий, Я.П. Троценко, Д.І. Черній «Технології чисельного моделювання. Лабораторний практикум» (Київ, 2024; розд. 7.1), презентацією «Лабораторна №1 (розділ — Технологія)» та поясненням викладача на парі 17.09.2026. Нумерація формул (7.1.x) — за посібником. Усі рисунки побудовано еталонною реалізацією з цього ж каталогу (mdo_lab1.py), яку перевірено на точному аналітичному розв'язку.

Зміст

1. Що треба зробити — коротко	2
2. Задача: фізика і математика	3
2.1. Фізичний зміст	3
2.2. Математична постановка (7.1.1)–(7.1.4)	3
2.3. Аналітичний розв’язок у термінах ТФКЗ	4
3. Крок 1. Контур	4
3.1. Вимоги до контуру	4
3.2. Дискретизація	4
4. Крок 2. Точки колокації та нормалі	5
5. Крок 3. Дискретний вихор і регуляризація	6
6. Крок 4. Система лінійних алгебраїчних рівнянь	7
7. Крок 5. Сітка «пікселів»	8
8. Крок 6. Поля швидкості, потенціалу і функції течії	8
8.1. Швидкість і функція течії	8
8.2. Потенціал і круговий арктангенс	9
8.3. Чому пряма формула для φ «не працює»	9
8.4. Перетворені формули (7.1.21)–(7.1.25')	10
9. Крок 7. Графіки	10
9.1. Як читати результат (і як зрозуміти, що все правильно)	12
10. Крок 8. Перевірка правильності	13
10.1. Швидкі тести	13
10.2. Тест на точному розв’язку: пластина	13
11. Типові помилки	14
12. Що включити у звіт	15
Додаток А. Повний код <code>mdo_lab1.py</code>	16
Додаток Б. Шаблон <code>run_lab1.py</code>	18
Джерела	19

1. Що треба зробити — коротко

Комплексна лабораторна з ТЧМ складається з двох частин: **лаб. №1** (стаціонарна течія — навчальна задача, щоб опанувати саму обчислювальну технологію) і **лаб. №2** (нестационарна відривна течія, розд. 7.2 посібника). Цей гайд — про лаб. №1. Мета, за словами викладача: на простій задачі з зрозумілим фізичним змістом («тренуємось на кішках») побачити, як працює технологія, які формули працюють, а які ні.

Вхідні дані (п. 7.1.3)

1. Контур перешкоди L_d — **свій у кожного** («ієрогліф»): впорядкований масив точок $\omega_{0j} = x_{0j} + iy_{0j}$, $j = 1, \dots, M$.
2. M — кількість дискретних особливостей (довільна; **алгоритм має підтримувати змінне M**).
3. Γ_0 — задана циркуляція навколо перешкоди (у презентації: $\Gamma_0 \in \{-1, 0, 1\}$).
4. $\vec{V}_\infty = (u_\infty, v_\infty) = (\cos \alpha, \sin \alpha)$, $|\vec{V}_\infty| = 1$.

Результат — 4 графіки (для кожного Γ_0)

1. векторне поле швидкостей $\vec{V}(x, y)$;
2. ізолії модуля швидкості $|\vec{V}| = \text{const}$;
3. ізолії потенціалу $\varphi(x, y) = \text{const}$;

4. ізолінії функції течії $\psi(x, y) = \text{const}$ (лінії течії).

Маршрут. Кожен крок нижче — одна-дві функції коду (повний код — у Додатку А).

Крок	Що робимо	Формули	Функція
1	Задаємо контур, нормуємо до розміру 1, розбиваємо на M точок	7.1.2	normalize, resample
2	Точки колокації та нормалі в них	7.1.2	collocation
3	Швидкість від одиничного дискретного вихору; регуляризація	7.1.17, 7.1.18	vortex_velocity
4	Складаємо і розв'язуємо СЛАР для $\Gamma_1, \dots, \Gamma_M$	7.1.19–7.1.20	solve_gammas
5	Сітка «пікселів» у координатах задачі	—	grid
6	Поля $\vec{V}$, φ , ψ — правильними (перетвореними) формулами	7.1.13–7.1.25'	velocity, phi_transformed, psi_transformed
7	Будуємо 4 графіки для $\Gamma_0 = -1, 0, 1$	—	run_lab1.py
8	Перевіряємо правильність (тести, збіжність)	—	plate_exact_velocity

2. Задача: фізика і математика

2.1. Фізичний зміст

Моделюємо **стаціонарну** (параметри в кожній точці не змінюються з часом) **плоску** (картина однакова вздовж осі z) течію ідеальної рідини навколо тонкої перешкоди — як течія навколо острова в протоці. Далеко від перешкоди потік однорідний: $\vec{V} \rightarrow \vec{V}_\infty$.

Течія **потенціальна**: існує функція $\varphi(x, y)$ — потенціал, така що

$$\vec{V} = \nabla \varphi = (u, v) = \left(\frac{\partial \varphi}{\partial x}, \frac{\partial \varphi}{\partial y} \right)$$

Тут ∇ — оператор Гамільтона (набла), $\Delta = \nabla^2$ — оператор Лапласа.

2.2. Математична постановка (7.1.1)–(7.1.4)

Зовнішня задача Неймана для рівняння Лапласа:

$$\Delta \varphi = \frac{\partial^2 \varphi}{\partial x^2} + \frac{\partial^2 \varphi}{\partial y^2} = 0, \quad \vec{r} \in D^+ \quad (7.1.1)$$

$$\left. \frac{\partial \varphi}{\partial n} \right|_{\vec{r} \in L_d} = 0 \quad \text{— умова непроникнення} \quad (7.1.2)$$

$$\oint_C \nabla \varphi \cdot d\vec{r} = \Gamma_0, \quad C \text{ охоплює } L_d \quad \text{— умова циркуляції} \quad (7.1.3)$$

$$\lim_{|\vec{r}| \rightarrow \infty} \nabla \varphi = \vec{V}_\infty \quad (7.1.4)$$

Навіщо (7.1.3). Перешкода «виризає дірку» в площині, тож область D^+ — **двозв'язна** (порядок зв'язності = кількість розрізів, що роблять область однозв'язною, + 1; тут потрібен один розріз від контуру до нескінченності). У двозв'язній області задача Неймана має розв'язок лише з точністю до циркуляції — її і фіксує Γ_0 . Цей розріз ще з'явиться у кроці 6 — як лінія розриву потенціалу.

2.3. Аналітичний розв'язок у термінах ТФКЗ

Характеристична функція $\Phi = \varphi + i\psi$ (ψ — функція течії) аналітична, φ і ψ пов'язані умовами Коші–Рімана $\varphi_x = \psi_y$, $\varphi_y = -\psi_x$. Розв'язок шукають у вигляді

$$\Phi(z) = z\bar{V}_\infty + \frac{1}{2\pi i} \int_{L_d} \gamma(\omega) \ln(z - \omega) d\omega + \text{const} \quad (7.1.5)$$

$$\bar{V}(z) = u - iv = \frac{d\Phi}{dz} = \bar{V}_\infty + \frac{1}{2\pi i} \int_{L_d} \frac{\gamma(\omega)}{z - \omega} d\omega \quad (7.1.6)$$

Невідома лише густина $\gamma(\omega)$ на контурі. Підстановка (7.1.6) у (7.1.2)–(7.1.3) дає **сингулярне інтегральне рівняння** (ядро типу Коші, інтеграл — у сенсі головного значення). Аналітично воно розв'язується лише для канонічних контурів (відрізок, дуга кола), тому застосовуємо чисельний **метод дискретних особливостей (МДО)**: неперервну густину на контурі замінюємо M точковими вихорами інтенсивностей Γ_j :

$$\Phi(z) = z\bar{V}_\infty + \sum_{j=1}^M \frac{\Gamma_j}{2\pi i} \ln(z - \omega_{0j}), \quad \Gamma_0 = \sum_{j=1}^M \Gamma_j \quad (7.1.5', 7.1.8')$$

Уся лабораторна зводиться до того, щоб знайти $\Gamma_1, \dots, \Gamma_M$ і правильно намалювати поля.

3. Крок 1. Контур

3.1. Вимоги до контуру

- **Розімкнена ламана/крива без самоперетинів**, зв'язна (з одного шматка). Гіллясті контури допустимі, але потребують окремих формул (додаток презентації) — для першої лаби беріть простий.
- **Розмір 1**: відстань між двома найвіддаленішими точками контуру дорівнює 1 (характерний розмір, «міделевий переріз»). Тоді перешкода вміщується в квадрат $[-0.5, 0.5]^2$, як і казав викладач.
- Центр — у початку координат.
- Точки впорядковані: ω_{01} — початок, ω_{0M} — кінець. Правило обходу теорії сингулярних рівнянь: область D^+ — **ліворуч** від напрямку обходу.

3.2. Дискретизація

Задаємо контур кількома вершинами (так зручно описати «ієрогліф»), нормуємо і розставляємо M точок майже рівномірно за довжиною дуги. **Кутові вершини мають потрапити в набір точок**, інакше дискретизація «зрізає кути» і форма перешкоди спотворюється.

```
def normalize(vertices):
    """Масштабує контур до характерного розміру 1 (найбільша відстань між точками) і центрує
```

```

в (0, 0)."""
P = np.asarray(vertices, float)
diff = P[:, None, :] - P[None, :, :]
P = P / np.sqrt((diff ** 2).sum(-1)).max()
return P - 0.5 * (P.min(0) + P.max(0))

```

```

def resample(vertices, M):
    """М точок ωj, майже рівномірно за довжиною дуги ламаної; кутові вершини ламаної
    зберігаються."""
    P = np.asarray(vertices, float)
    s = np.concatenate([[0.0], np.cumsum(np.hypot(*np.diff(P, axis=0).T))])
    t = np.linspace(0.0, s[-1], M)
    for si in s[1:-1]:
        t[np.argmax(np.abs(t - si))] = si
    return np.column_stack([np.interp(t, s, P[:, 0]), np.interp(t, s, P[:, 1])])

```

Порада

Крок між точками $h \approx L/(M-1)$, де L — довжина контуру. Для контуру розміру 1 довжиною 2–3 та $M = 40 \dots 80$ маємо $h \approx 0.03 \dots 0.07$. Запам'ятайте h — він знадобиться для вибору параметра регуляризації.

4. Крок 2. Точки колокації та нормалі

Між кожними двома сусідніми особливостями ставимо **точку колокації** — у ній вимагатимемо непроникнення. Колокацій **рівно на одну менше**, ніж особливостей: $M-1$.

$$x_k = \frac{x_{0k} + x_{0k+1}}{2}, \quad y_k = \frac{y_{0k} + y_{0k+1}}{2}, \quad k = 1, \dots, M-1 \quad (7.1.2)$$

Одинична нормаль у точці колокації — поворот вектора відрізка на $+90^\circ$ (дивиться ліворуч від обходу):

$$n_{xk} = -\frac{y_{0k+1} - y_{0k}}{\ell_k}, \quad n_{yk} = \frac{x_{0k+1} - x_{0k}}{\ell_k}, \quad \ell_k = \sqrt{(x_{0k+1} - x_{0k})^2 + (y_{0k+1} - y_{0k})^2} \quad (7.1.2)$$

```

def collocation(P):
    """Точки колокації (між сусідніми особливостями) та одиничні нормалі в них, (7.1.2)."""
    C = 0.5 * (P[:-1] + P[1:])
    d = np.diff(P, axis=0)
    N = np.column_stack([-d[:, 1], d[:, 0]]) / np.hypot(d[:, 0], d[:, 1])[:, None]
    return C, N

```

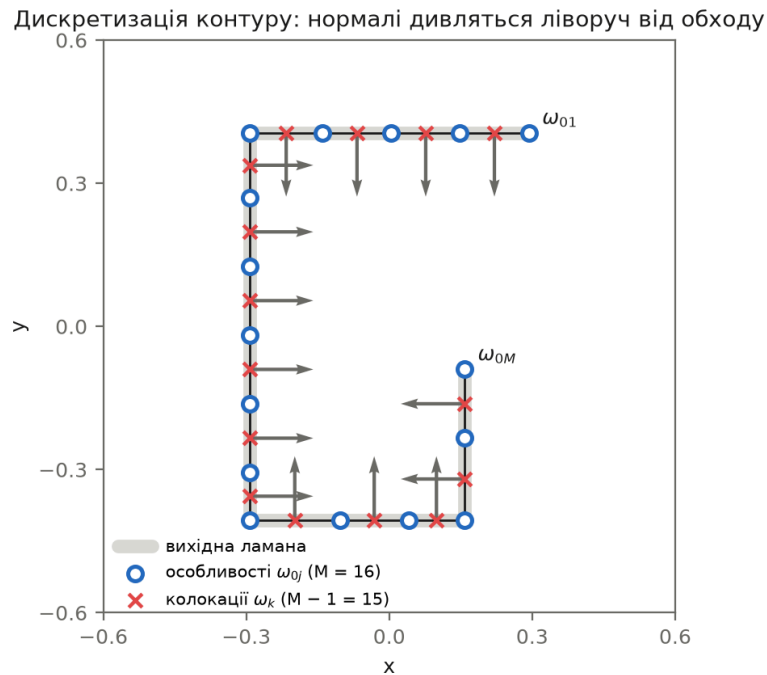


Рисунок 1: Дискретизація «ієрогліфа» при $M = 16$: кружки — дискретні особливості ω_{0j} (зокрема в кутах), хрестики — точки колокації, стрілки — нормалі. Порівняйте з рис. 7.1.3 посібника.

Корисно знати

Рівняння непроникнення (крок 4) не змінюються при $\vec{n} \rightarrow -\vec{n}$ і при масштабуванні $\vec{n}$, тому для лаб. №1 знак нормалі на результат не впливає. Проте нормаль має бути саме **нормаллю** (не дотичною!), а правило « D^+ ліворуч» знадобиться в лаб. №2 та для інтерпретації Γ_j .

5. Крок 3. Дискретний вихор і регуляризація

Швидкість, яку індукує в точці (x, y) вихор одиничної інтенсивності, розміщений в ω_{0j} :

$$\vec{V}_j(x, y, x_{0j}, y_{0j}) = \left(\frac{y_{0j} - y}{2\pi R_j^2}, \frac{x - x_{0j}}{2\pi R_j^2} \right) \quad (7.1.17)$$

При $(x, y) \rightarrow \omega_{0j}$ знаменник прямує до нуля — звідси назва «дискретні **особливості**». Щоб поля скрізь були скінченні, відстань обмежують знизу **параметром регуляризації** r_j :

$$R_j = \max \left\{ r_j, \sqrt{(x - x_{0j})^2 + (y - y_{0j})^2} \right\}, \quad r_j \approx 0.01 \quad (7.1.18)$$

```
def vortex_velocity(x, y, x0, y0, delta=0.0):
    """V_j з (7.1.17); при delta > 0 — регуляризований варіант (7.1.17'), (7.1.18)."""
    dx, dy = x - x0, y - y0
    R2 = np.maximum(dx * dx + dy * dy, delta * delta)
    return -dy / (TWO_PI * R2), dx / (TWO_PI * R2)
```

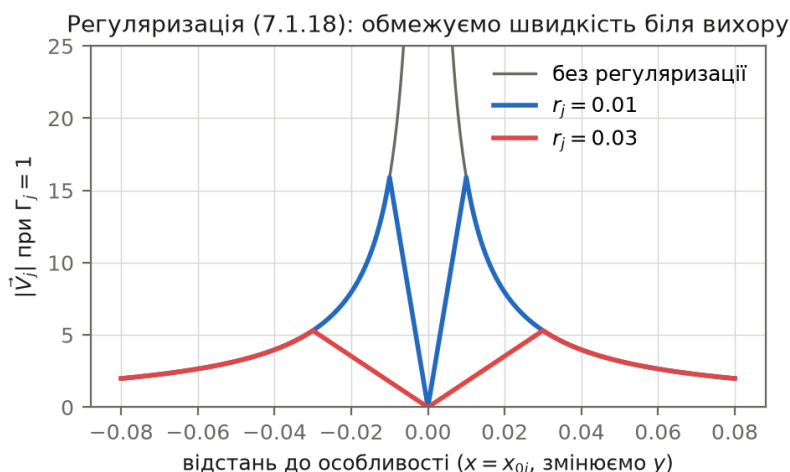


Рисунок 2: Модуль швидкості одиничного вихору вздовж прямої через нього: всередині кола радіуса r_j швидкість обмежена і лінійно спадає до нуля.

Регуляризація — лише для візуалізації

Відстань від точки колокації до найближчої особливості дорівнює $h/2$. Якщо використати регуляризовану формулу в матриці СЛАР і взяти $r_j > h/2$ (наприклад, $r_j = 0.01$ при $M = 150$), коефіцієнти матриці спотворяться і розв'язок стане неправильним. Тому в кроці 4 беремо формулу (7.1.17) без регуляризації (колокації ніколи не збігаються з особливостями), а (7.1.18) застосовуємо лише при обчисленні полів на сітці. Орієнтир: $r_j \leq h/2$.

6. Крок 4. Система лінійних алгебраїчних рівнянь

Непроникнення (7.1.2) вимагаємо в кожній з $M - 1$ точок колокації, а циркуляцію (7.1.3) — одним додатковим рівнянням. Разом — M рівнянь на M невідомих:

$$\sum_{j=1}^M \Gamma_j (\vec{n}_k \cdot \vec{V}_j(x_k, y_k, x_{0j}, y_{0j})) = -\vec{n}_k \cdot \vec{V}_\infty, \quad k = 1, \dots, M - 1 \quad (7.1.19)$$

$$\sum_{j=1}^M \Gamma_j = \Gamma_0 \quad (7.1.20)$$

У матричному вигляді $A\Gamma = b$:

$$A_{kj} = n_{xk} u_j(x_k, y_k) + n_{yk} v_j(x_k, y_k), \quad b_k = -(n_{xk} u_\infty + n_{yk} v_\infty), \quad A_{Mj} = 1, \quad b_M = \Gamma_0.$$

Матриця заповнена (кожен вихор впливає на кожну колокацію), розмір невеликий — підходить звичайний метод Гаусса (`numpy.linalg.solve`).

```
def solve_gammas(P, v_inf, gamma0):
    """Розв'язує систему (7.1.19)-(7.1.20) відносно  $\Gamma_1.. \Gamma_M$ ."""
    M = len(P)
    C, N = collocation(P)
    u, v = vortex_velocity(C[:, [0]], C[:, [1]], P[None, :, 0], P[None, :, 1])
    A = np.empty((M, M))
    b = np.empty(M)
    A[:-1] = N[:, [0]] * u + N[:, [1]] * v
    b[:-1] = -N @ np.asarray(v_inf, float)
    A[-1] = 1.0
```

```
b[-1] = gamma0
return np.linalg.solve(A, b)
```

Контроль після розв'язання

- $\sum \Gamma_j = \Gamma_0$ з машинною точністю;
- $\max_k |\vec{n}_k \cdot \vec{V}(x_k, y_k)| \approx 10^{-15}$ — непроникнення в колокаціях виконане;
- якщо solve скаржиться на вироджену матрицю — у контурі є точки, що збігаються (нульовий відрізок).

7. Крок 5. Сітка «пікселів»

Порада викладача: будь-яке зображення — це масив пікселів. Реальний розмір пікселя на екрані нас не цікавить; важливо, що **кожен піксель має координати в системі, пов'язаній з перешкодою**. Беремо зображення $N_x \times N_y$ (наприклад 400×400) і прямокутник $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$ у координатах задачі (перешкода в $[-0.5, 0.5]^2$, область — наприклад $[-1, 1]^2$). Центр пікселя (i, j) :

$$x_i = x_{\min} + \left(i + \frac{1}{2}\right) \frac{x_{\max} - x_{\min}}{N_x}, \quad y_j = y_{\min} + \left(j + \frac{1}{2}\right) \frac{y_{\max} - y_{\min}}{N_y}.$$

Далі в кожному пікселі обчислюємо φ , ψ або $|\vec{V}|$ і перетворюємо значення на колір. З пилу це робиться векторно, без циклу по пікселях:

```
def grid(n=400, lim=1.0):
    """Сітка «пікселів»: кожному пікселю (i, j) відповідає точка (x, y) у координатах
    задачі."""
    s = np.linspace(-lim, lim, n)
    return np.meshgrid(s, s)
```

Увага

На екрані вісь y зазвичай іде **вниз**. У matplotlib.imshow задавайте origin="lower", інакше картинка перевернеться. np.meshgrid(x, y) повертає масиви форми (N_y, N_x) — рядок = фіксоване y .

8. Крок 6. Поля швидкості, потенціалу і функції течії

8.1. Швидкість і функція течії

$$\vec{V}(x, y) = \vec{V}_{\infty} + \sum_{j=1}^M \Gamma_j \vec{V}_j(x, y, x_{0j}, y_{0j}) \quad (7.1.16')$$

$$\psi(x, y) = y u_{\infty} - x v_{\infty} - \sum_{j=1}^M \frac{\Gamma_j}{2\pi} \ln R_j \quad (7.1.15')$$

```
def velocity(X, Y, P, G, v_inf, delta):
    """Векторне поле (7.1.16')."""
    U = np.full_like(X, v_inf[0])
    V = np.full_like(X, v_inf[1])
    for (x0, y0), g in zip(P, G):
        du, dv = vortex_velocity(X, Y, x0, y0, delta)
```

```

U += g * du
V += g * dv
return U, V

```

8.2. Потенціал і круговий арктангенс

Дійсна частина $\frac{\Gamma_j}{2\pi i} \ln(z - \omega_{0j})$ — це $\Gamma_j \arg(z - \omega_{0j}) / (2\pi)$. Тому

$$\varphi(x, y) = xu_\infty + yv_\infty + \sum_{j=1}^M \Gamma_j \theta_j(x, y), \quad \theta_j = \frac{1}{2\pi} \arg(z - \omega_{0j}) \in [0, 1) \quad (7.1.13', 7.1.14')$$

Кут θ_j — **круговий арктангенс**: кут, під яким точку (x, y) видно з ω_{0j} , відлічений у межах повного оберту. Посібник розписує його через $\arctg$ по чвертях; у коді це $\text{atan2}(y - y_0, x - x_0)$, зведений до $[0, 2\pi)$.

```

def theta(x, y, x0, y0):
    """Круговий арктангенс (7.1.14): arg(z - w0)/(2π) ∈ [0, 1)."""
    return np.mod(np.arctan2(y - y0, x - x0), TWO_PI) / TWO_PI

```

Звичайний арктангенс не працює

$\text{atan}((y - y_0)/(x - x_0))$ повертає значення лише в $(-\pi/2, \pi/2)$ і плутає протилежні чверті — потенціал виходить неправильним у половині площини. Потрібен саме atan2 (круговий арктангенс).

8.3. Чому пряма формула для φ «не працює»

Навіть із круговим арктангенсом кожен доданок θ_j має **розрив** на промені, що йде від ω_{0j} (у нашій реалізації — вправо, де $\arg$ стрибає з 2π до 0). Потенціал — багатозначна функція у двов'язній області, і пряма формула (7.1.13') дає M променів-розрізів, по одному від кожної особливості, зі стрибками Γ_j . Ізолінії φ через це виходять «зламаними» (рис. нижче, ліворуч).

Потенціал $\varphi(x, y)$, $\Gamma_0 = 1$

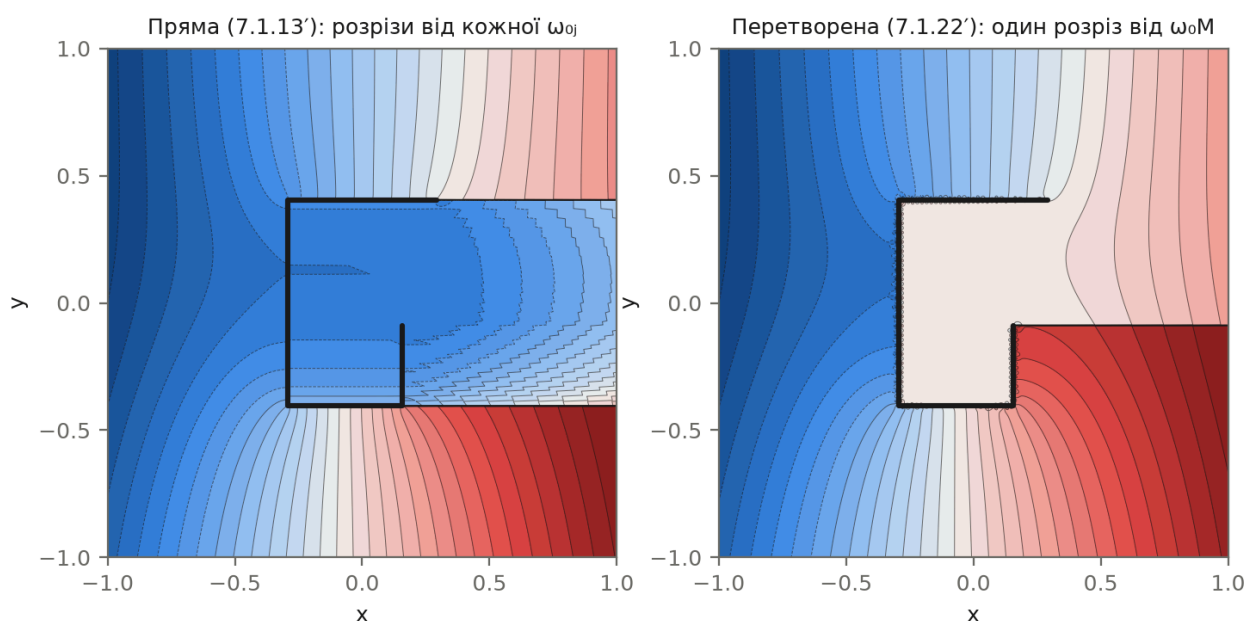


Рисунок 3: Потенціал при $\Gamma_0 = 1$. Ліворуч — пряма формула: віяло розрізів праворуч від контуру. Праворуч — перетворена формула: лишається **один** розріз від ω_{0M} зі стрибком Γ_0 .

8.4. Перетворені формули (7.1.21)–(7.1.25')

Підсумовування частинами (перетворення Абеля) з $S_j = \sum_{k=1}^j \Gamma_k$, $S_M = \Gamma_0$ та $\ln(z - \omega_{0j+1}) - \ln(z - \omega_{0j}) \approx -\frac{\omega_{0j+1} - \omega_{0j}}{z - \zeta_j}$ замінює ланцюжок вихорів на **диполі** в середині відрізків $\zeta_j = (\omega_{0j} + \omega_{0j+1})/2$ плюс один вихор Γ_0 у кінцевій точці:

$$\Phi(z) = z\bar{V}_\infty + \sum_{j=1}^{M-1} \frac{(\omega_{0j+1} - \omega_{0j})S_j}{2\pi i(z - \zeta_j)} + \frac{\Gamma_0}{2\pi i} \ln(z - \omega_{0M}) \quad (7.1.21)$$

У дійсних змінних (x_j, y_j – координати ζ_j , R_j – регуляризована відстань до ζ_j):

$$\varphi = xu_\infty + yv_\infty + \Gamma_0\theta_M + \sum_{j=1}^{M-1} \frac{S_j}{2\pi} \frac{(y_{0j+1} - y_{0j})(x - x_j) - (x_{0j+1} - x_{0j})(y - y_j)}{R_j^2} \quad (7.1.22')$$

$$\psi = yu_\infty - xv_\infty - \frac{\Gamma_0}{2\pi} \ln R_M - \sum_{j=1}^{M-1} \frac{S_j}{2\pi} \frac{(x_{0j+1} - x_{0j})(x - x_j) + (y_{0j+1} - y_{0j})(y - y_j)}{R_j^2} \quad (7.1.23')$$

Диполь — однозначна функція, тож розрізів від них немає. Лишається один розріз вихору Γ_0 — це і є той самий розріз, що робить двозв'язну область однозв'язною (розд. 2.2). При $\Gamma_0 = 0$ розрізів немає взагалі.

```
def phi_transformed(X, Y, P, G, v_inf, delta):
    """Перетворений потенціал (7.1.22'): диполі + один вихор Г0 в ω0M — один розріз замість М."""
    gamma0 = G.sum()
    F = X * v_inf[0] + Y * v_inf[1] + gamma0 * theta(X, Y, *P[-1])
    for Sj, (xj, yj), (dxj, dyj) in zip(*_dipoles(P, G)):
        R2 = np.maximum((X - xj) ** 2 + (Y - yj) ** 2, delta * delta)
        F += Sj / TWO_PI * (dyj * (X - xj) - dxj * (Y - yj)) / R2
    return F
```

```
def psi_transformed(X, Y, P, G, v_inf, delta):
    """Перетворена функція течії (7.1.23')."""
    gamma0 = G.sum()
    RM = np.maximum(np.hypot(X - P[-1, 0], Y - P[-1, 1]), delta)
    F = Y * v_inf[0] - X * v_inf[1] - gamma0 / TWO_PI * np.log(RM)
    for Sj, (xj, yj), (dxj, dyj) in zip(*_dipoles(P, G)):
        R2 = np.maximum((X - xj) ** 2 + (Y - yj) ** 2, delta * delta)
        F -= Sj / TWO_PI * (dxj * (X - xj) + dyj * (Y - yj)) / R2
    return F
```

Два нюанси

- У (7.1.23') посібника в доданку з Γ_0 стоїть $\ln R_j^2$; за нерегуляризованою формулою (7.1.23) там має бути $\ln R_M$ — логарифм відстані до ω_{0M} . Схоже на друкарську помилку; у коді — $\ln R_M$.
- Стрибок φ **через сам контур** (усередині «гачка» на рис. вище) — не артефакт: контур — вихровий шар, на ньому дотична швидкість, а з нею й потенціал, справді розривні. Артефактами є лише розрізи-промені у вільному потоці.

9. Крок 7. Графіки

Для кожного значення $\Gamma_0 \in \{-1, 0, 1\}$ будемо чотири картини. Мінімальний шаблон — `gun_lab1.py` (Додаток Б): у ньому достатньо замінити контур і параметри.

Результат лаб. №1: $M = 60$, $\Gamma_0 = 0$, $\vec{V}_\infty = (1, 0)$, $r_j = 0.01$

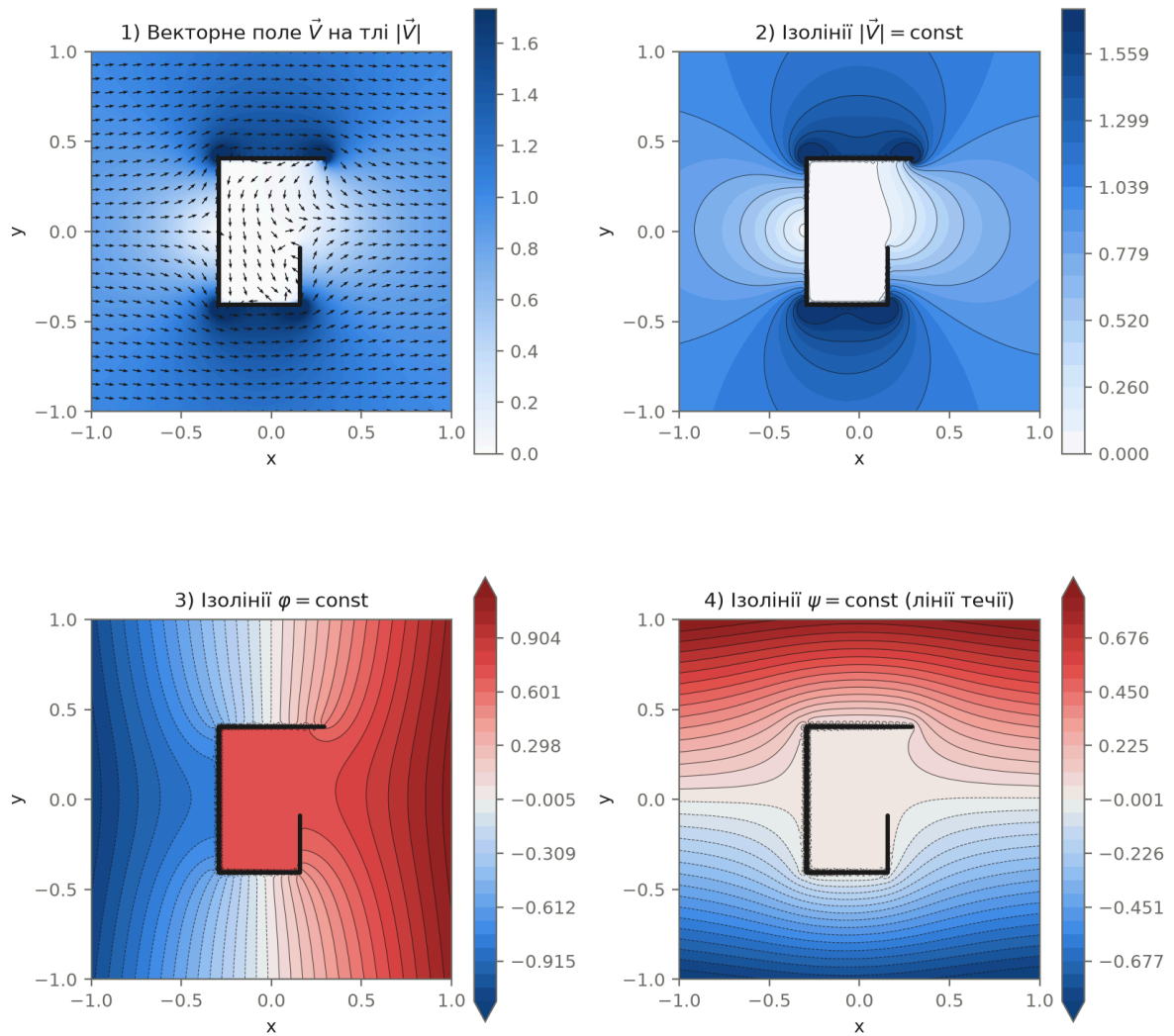
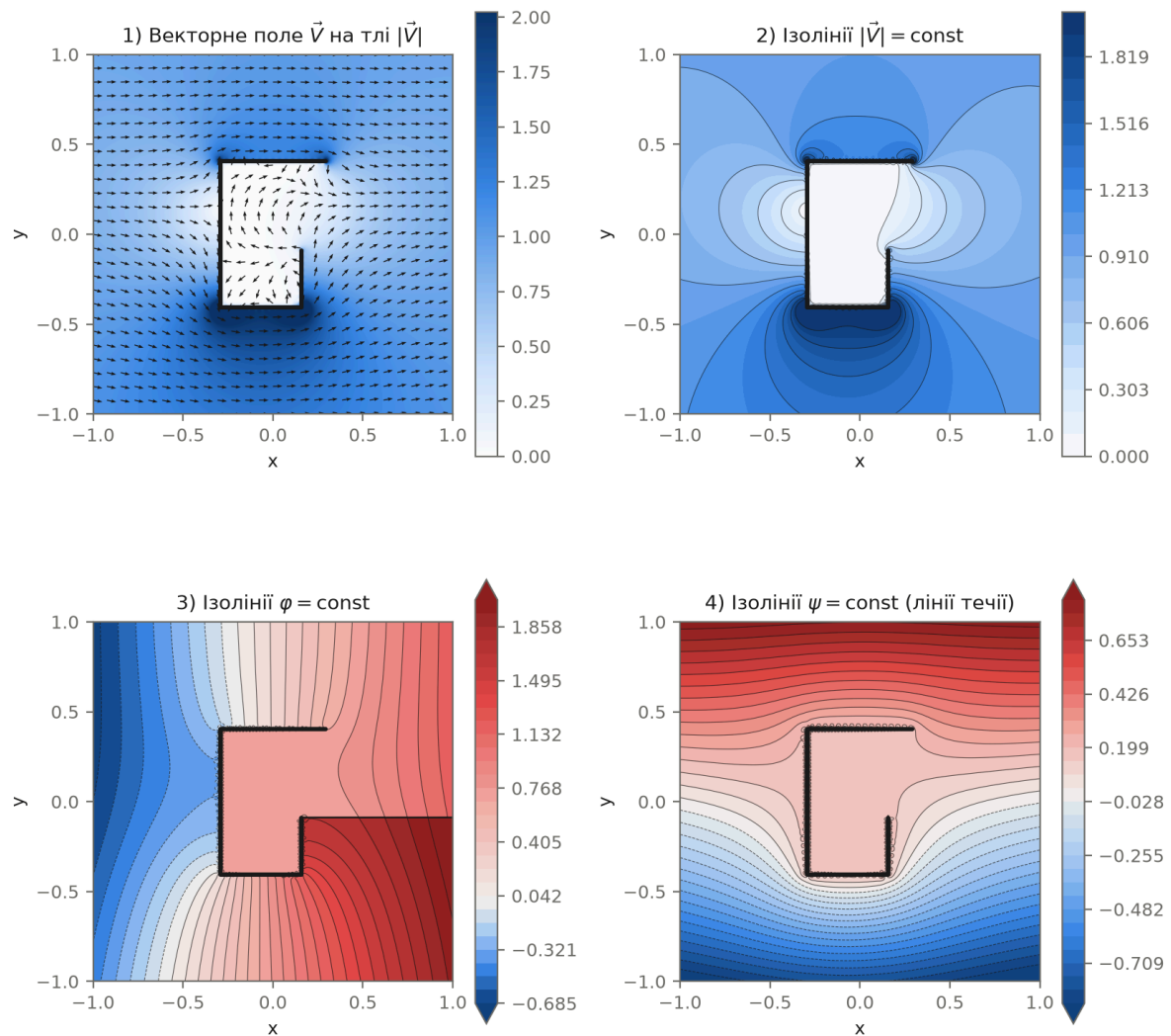
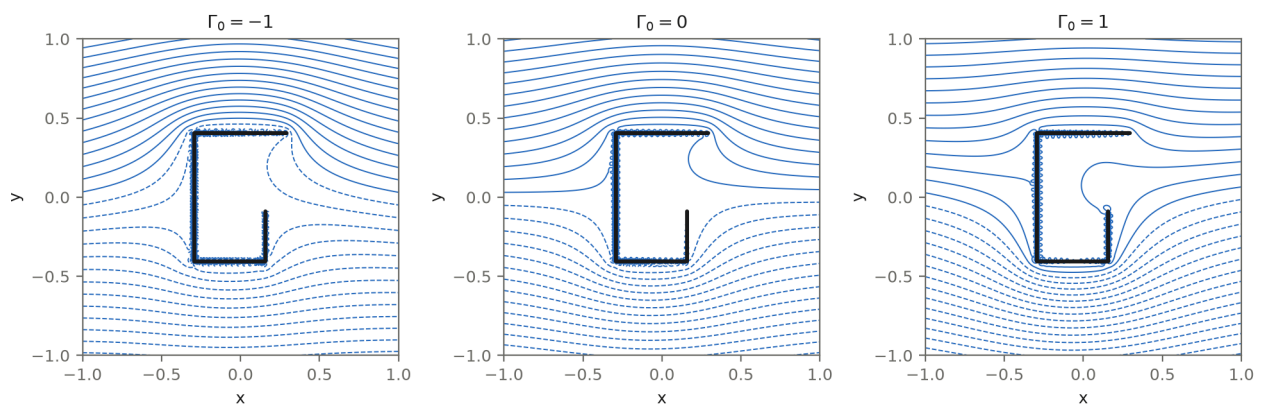


Рисунок 4: Результат для «ієрогліфа»: $M = 60$, $\Gamma_0 = 0$, $\vec{V}_\infty = (1, 0)$, $r_j = 0.01$.

Результат лаб. №1: $M = 60$, $\Gamma_0 = 1$, $\vec{V}_\infty = (1, 0)$, $r_j = 0.01$ Рисунок 5: Те саме при $\Gamma_0 = 1$.Лінії течії $\psi = \text{const}$ при різних циркуляціяхРисунок 6: Лінії течії при $\Gamma_0 = -1, 0, 1$ (пунктир — від'ємні значення ψ).

9.1. Як читати результат (і як зрозуміти, що все правильно)

- Далеко від перешкоди стрілки однакові, $|\vec{V}| \approx 1$, ізолінії ψ — майже горизонтальні прямі.

- **Біля вільних кінців контуру** ω_{01}, ω_{0M} швидкість різко зростає: розв'язок має особливості на обох кінцях — саме такий клас розв'язків дає схема « M особливостей, $M - 1$ колокацій».
- **У заглибленні** («всередині гачка») течія повільна — застійна зона, $|\vec{V}| \approx 0$.
- **Лінії течії не перетинають контур**; сам контур лежить на лінії $\psi = \text{const}$.
- Ізолії φ і ψ **ортогональні** одна одній (наслідок умов Коші–Рімана).
- **Вплив циркуляції**: при $\Gamma_0 > 0$ (проти годинникової стрілки) і потоці зліва направо течія під перешкодою прискорюється, над нею — сповільнюється; при $\Gamma_0 < 0$ — навпаки. Критичні точки зсуваються.

Оформлення графіків

Для $|\vec{V}|$ — послідовна шкала одного кольору (світле = повільно). Для φ і ψ — розбіжна шкала з нейтральним кольором у нулі. Завжди малюйте сам контур поверх полів, підписуйте осі й значення M , Γ_0 , $\vec{V}_\infty$ у заголовку. Векторне поле — на рідшій сітці ($\approx 25 \times 25$), інакше стрілки зливаються.

10. Крок 8. Перевірка правильності

10.1. Швидкі тести

Перевірка	Очікування
$\sum \Gamma_j$	$= \Gamma_0$
$ \vec{n}_k \cdot \vec{V} $ у колокаціях	$\sim 10^{-15}$
$ \vec{V} $ на краю області $[-1, 1]^2$	$\rightarrow 1$ при віддаленні
ψ у точках, зміщених від колокацій на $0.02\vec{n}$	майже стала вздовж контуру
симетричний контур, $\Gamma_0 = 0$, $\vec{V}_\infty$ уздовж осі симетрії	симетрична картина
подвоєння M	картина майже не змінюється

10.2. Тест на точному розв'язку: пластина

Для відрізка (пластини) аналітичний розв'язок відомий. Для пластини $[-a, a]$ на дійсній осі в потоці $\vec{V}_\infty = (\cos \alpha, \sin \alpha)$ з циркуляцією Γ :

$$\frac{dw}{dz} = \cos \alpha - i \sin \alpha \frac{z}{\sqrt{z^2 - a^2}} + \frac{\Gamma}{2\pi i \sqrt{z^2 - a^2}}, \quad \sqrt{z^2 - a^2} \sim z \text{ при } z \rightarrow \infty.$$

(У коді корінь рахується як $\sqrt{z-a}\sqrt{z+a}$ — це дає потрібну вітку з розрізом по пластині.) Для вертикальної пластини в горизонтальному потоці повертаємо систему координат: $z' = -iz$, $\alpha = -\pi/2$, $\bar{V}(z) = -i \cdot (dw)/(dz')$.

```
def plate_exact_velocity(X, Y, a, gamma):
    """Точна швидкість для вертикальної пластини [-ia, ia] у потоці  $V_\infty = (1, 0)$  з
    циркуляцією gamma.

    У системі пластини  $z' = -iz$  (пластина на  $[-a, a]$ , потік під кутом  $-\pi/2$ ):
     $dw/dz' = \cos \alpha - i \sin \alpha \cdot z' / \sqrt{(z')^2 - a^2} + \Gamma / (2\pi i \sqrt{(z')^2 - a^2})$ .
    """
    zp = -1j * (X + 1j * Y)
    root = np.sqrt(zp - a) * np.sqrt(zp + a)
    alpha = -np.pi / 2
```

```
with np.errstate(divide="ignore", invalid="ignore"):
    dw = np.cos(alpha) - 1j * np.sin(alpha) * zp / root + gamma / (2j * np.pi * root)
    conj_v = -1j * dw
    return conj_v.real, -conj_v.imag
```

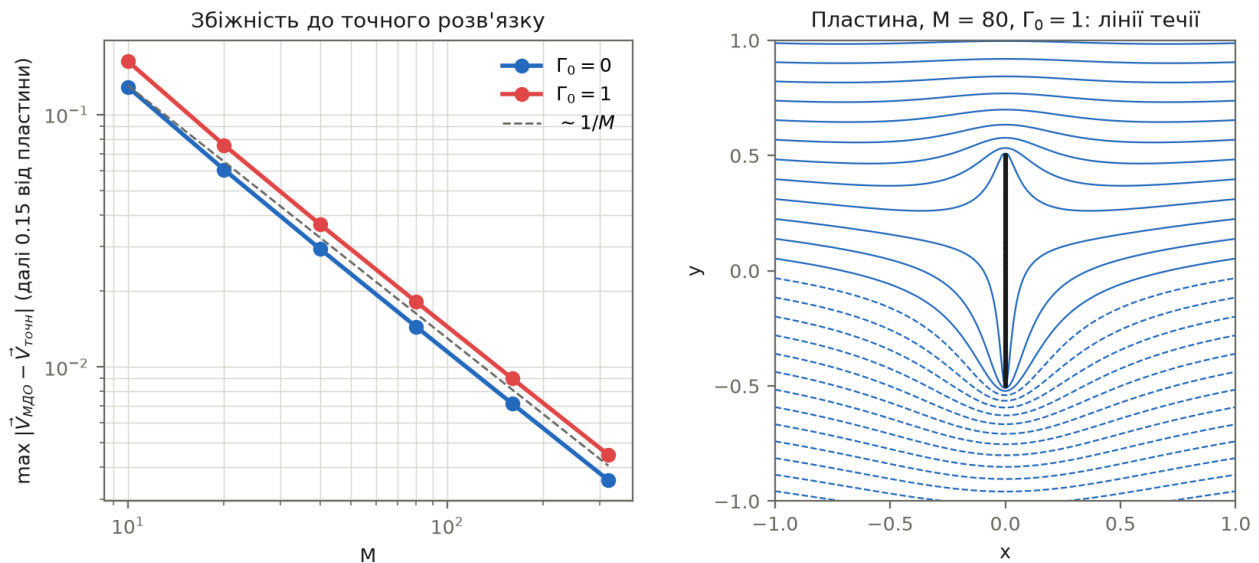


Рисунок 7: Ліворуч: максимальна похибка швидкості МДО відносно точного розв'язку (на відстані > 0.15 від пластины) спадає як $1/M$ — перший порядок збіжності. Праворуч: лінії течії навколо пластины при $\Gamma_0 = 1$.

Якщо ваша реалізація на цьому тесті **не збігається** до точного розв'язку зі зростанням M — шукайте помилку до того, як будувати графіки для свого контуру. Найчастіше винні знак у $\vec{V}_j$ або пропущений рядок (7.1.20).

11. Типові помилки

№	Симптом	Причина / виправлення
1	Потенціал неправильний у лівій півплощині	atan замість кругового арктангенса $\rightarrow$ atan2 зведений до $[0, 2\pi)$
2	Ізолінії φ «ламаються» віялом променів	Пряма формула (7.1.13') $\rightarrow$ перетворена (7.1.22')
3	Ділення на нуль / NaN у матриці	Колокації збіглися з особливостями або в контурі дубльовані точки
4	$\sum \Gamma_j \neq \Gamma_0$, «дивна» картина	Забули рядок (7.1.20); розв'язують $M - 1$ рівнянь на M невідомих
5	Результат залежить від M і погіршується при великих M	Регуляризація в матриці з $r_j > h/2$ — див. крок 3
6	Течія «протікає» крізь контур	Замість нормалі взято дотичну; або неправильно рахуються колокації
7	Перешкода крихітна / величезна на графіку	Контур не нормовано до розміру 1
8	Картинка перевернута / розтягнута	imshow без origin="lower"; не задано set_aspect("equal")

9	Гострі «шпильки» швидкості біля контуру	Нормально поблизу кінців; всюди — зменште крок сітки або збільште r_j (до $h/2$)
10	Контур «зрізає кути»	Рівномірне розбиття без збереження вершин — див. крок 1

12. Що включити у звіт

Рекомендація

У записі пари вимоги до звіту не озвучувались — нижче розумна структура; уточніть у викладача формат здачі.

1. Титул, мета роботи.
2. Постановка задачі: (7.1.1)–(7.1.4), фізичний зміст.
3. Метод: дискретизація контуру, точки колокації й нормалі, СЛАР (7.1.19)–(7.1.20), регуляризація, перетворені формули для φ і ψ (із поясненням, навіщо вони).
4. Вхідні дані: рисунок контуру з особливостями і колокаціями; M , Γ_0 , $\vec{V}_\infty$, r_j , область, роздільність.
5. Результати: чотири графіки для кожного $\Gamma_0 \in \{-1, 0, 1\}$; коротке пояснення картин.
6. Перевірка: $\sum \Gamma_j$, нев'язка непроникнення, тест на пластині, порівняння двох значень M .
7. Висновки.
8. Додаток: код програми.

Додаток А. Повний код `mdo_lab1.py`

```

"""Лабораторна №1 ТЧМ: стаціонарна течія навколо перешкоди методом дискретних особливостей
(МДО).

Нумерація формул – за посібником Довгий, Троценко, Черній (2024), розд. 7.1.
"""

import numpy as np

TWO_PI = 2.0 * np.pi

# ----- Крок 1. Контур -----

def normalize(vertices):
    """Масштабує контур до характерного розміру 1 (найбільша відстань між точками) і центрує
    в (0, 0)."""
    P = np.asarray(vertices, float)
    diff = P[:, None, :] - P[None, :, :]
    P = P / np.sqrt((diff ** 2).sum(-1)).max()
    return P - 0.5 * (P.min(0) + P.max(0))

def resample(vertices, M):
    """М точок  $\omega_j$ , майже рівномірно за довжиною дуги ламаної; кутові вершини ламаної
    зберігаються."""
    P = np.asarray(vertices, float)
    s = np.concatenate([[0.0], np.cumsum(np.hypot(*np.diff(P, axis=0).T))])
    t = np.linspace(0.0, s[-1], M)
    for si in s[1:-1]:
        t[np.argmin(np.abs(t - si))] = si
    return np.column_stack([np.interp(t, s, P[:, 0]), np.interp(t, s, P[:, 1])])

# ----- Крок 2. Точки колокації та нормалі -----

def collocation(P):
    """Точки колокації (між сусідніми особливостями) та одиничні нормалі в них, (7.1.2)."""
    C = 0.5 * (P[:-1] + P[1:])
    d = np.diff(P, axis=0)
    N = np.column_stack([-d[:, 1], d[:, 0]]) / np.hypot(d[:, 0], d[:, 1])[ :, None]
    return C, N

# ----- Крок 3. Швидкість від одиничного вихору -----

def vortex_velocity(x, y, x0, y0, delta=0.0):
    """ $V_j$  з (7.1.17); при  $\delta > 0$  – регуляризований варіант (7.1.17'), (7.1.18)."""
    dx, dy = x - x0, y - y0
    R2 = np.maximum(dx * dx + dy * dy, delta * delta)
    return -dy / (TWO_PI * R2), dx / (TWO_PI * R2)

# ----- Крок 4. СЛАР -----

def solve_gammas(P, v_inf, gamma0):
    """Розв'язує систему (7.1.19)-(7.1.20) відносно  $\Gamma_1.. \Gamma_M$ ."""
    M = len(P)
    C, N = collocation(P)
    u, v = vortex_velocity(C[:, [0]], C[:, [1]], P[None, :, 0], P[None, :, 1])

```

```

A = np.empty((M, M))
b = np.empty(M)
A[:-1] = N[:, [0]] * u + N[:, [1]] * v
b[:-1] = -N @ np.asarray(v_inf, float)
A[-1] = 1.0
b[-1] = gamma0
return np.linalg.solve(A, b)

# ----- Крок 5-6. Поля на сітці -----

def grid(n=400, lim=1.0):
    """Сітка «пікселів»: кожному пікселю (i, j) відповідає точка (x, y) у координатах
    задачі."""
    s = np.linspace(-lim, lim, n)
    return np.meshgrid(s, s)

def theta(x, y, x0, y0):
    """Круговий арктангенс (7.1.14):  $\arg(z - \omega_0)/(2\pi) \in [0, 1)$ ."""
    return np.mod(np.arctan2(y - y0, x - x0), TWO_PI) / TWO_PI

def velocity(X, Y, P, G, v_inf, delta):
    """Векторне поле (7.1.16')."""
    U = np.full_like(X, v_inf[0])
    V = np.full_like(X, v_inf[1])
    for (x0, y0), g in zip(P, G):
        du, dv = vortex_velocity(X, Y, x0, y0, delta)
        U += g * du
        V += g * dv
    return U, V

def phi_direct(X, Y, P, G, v_inf):
    """Потенціал за прямою формулою (7.1.13'): має розрізи-промені від кожної
    особливості."""
    F = X * v_inf[0] + Y * v_inf[1]
    for (x0, y0), g in zip(P, G):
        F += g * theta(X, Y, x0, y0)
    return F

def psi_direct(X, Y, P, G, v_inf, delta):
    """Функція течії (7.1.15')."""
    F = Y * v_inf[0] - X * v_inf[1]
    for (x0, y0), g in zip(P, G):
        F -= g / TWO_PI * np.log(np.maximum(np.hypot(X - x0, Y - y0), delta))
    return F

def _dipoles(P, G):
    S = np.cumsum(G)[-1]
    mid = 0.5 * (P[:-1] + P[1:])
    d = np.diff(P, axis=0)
    return S, mid, d

def phi_transformed(X, Y, P, G, v_inf, delta):
    """Перетворений потенціал (7.1.22'): диполі + один вихор  $\Gamma_0$  в  $\omega_0 M$  — один розріз замість
     $M$ ."""

```

```

gamma0 = G.sum()
F = X * v_inf[0] + Y * v_inf[1] + gamma0 * theta(X, Y, *P[-1])
for Sj, (xj, yj), (dxj, dyj) in zip(*_dipoles(P, G)):
    R2 = np.maximum((X - xj) ** 2 + (Y - yj) ** 2, delta * delta)
    F += Sj / TWO_PI * (dyj * (X - xj) - dxj * (Y - yj)) / R2
return F

def psi_transformed(X, Y, P, G, v_inf, delta):
    """Перетворена функція течії (7.1.23')."""
    gamma0 = G.sum()
    RM = np.maximum(np.hypot(X - P[-1, 0], Y - P[-1, 1]), delta)
    F = Y * v_inf[0] - X * v_inf[1] - gamma0 / TWO_PI * np.log(RM)
    for Sj, (xj, yj), (dxj, dyj) in zip(*_dipoles(P, G)):
        R2 = np.maximum((X - xj) ** 2 + (Y - yj) ** 2, delta * delta)
        F -= Sj / TWO_PI * (dxj * (X - xj) + dyj * (Y - yj)) / R2
    return F

# ----- Перевірка: пластина, аналітичний розв'язок -----

def plate_exact_velocity(X, Y, a, gamma):
    """Точна швидкість для вертикальної пластини [-ia, ia] у потоці  $V_\infty = (1, 0)$  з циркуляцією  $\gamma$ .

    У системі пластини  $z' = -iz$  (пластина на  $[-a, a]$ , потік під кутом  $-\pi/2$ ):
     $dw/dz' = \cos \alpha - i \sin \alpha \cdot z' / \sqrt{(z')^2 - a^2} + \Gamma / (2\pi i \sqrt{(z')^2 - a^2})$ .
    """
    zp = -1j * (X + 1j * Y)
    root = np.sqrt(zp - a) * np.sqrt(zp + a)
    alpha = -np.pi / 2
    with np.errstate(divide="ignore", invalid="ignore"):
        dw = np.cos(alpha) - 1j * np.sin(alpha) * zp / root + gamma / (2j * np.pi * root)
    conj_v = -1j * dw
    return conj_v.real, -conj_v.imag

```

Додаток Б. Шаблон run_lab1.py

Замініть CONTOUR, M, GAMMA0, ALPHA — і отримаєте файл lab1_G<Г0>.png з чотирма графіками. Рисунки цього гайду буде make_figures.py.

```

"""Шаблон виконання лаб. №1: задайте свій контур, M, Г0, V∞ — отримаєте 4 графіки завдання."""
import matplotlib.pyplot as plt
import numpy as np

from mdo_lab1 import (collocation, grid, normalize, phi_transformed, psi_transformed,
                      resample, solve_gammas,
                      velocity)

# ---- Вхідні дані (п. 7.1.3) ----
CONTOUR = [(0.35, 0.45), (-0.3, 0.45), (-0.3, -0.45), (0.2, -0.45), (0.2, -0.1)] # свій «ієрогліф»
M = 60 # кількість дискретних особливостей
GAMMA0 = 1.0 # циркуляція: запустіть для -1, 0, 1
ALPHA = 0.0 # кут набігаючого потоку, рад
V_INF = np.array([np.cos(ALPHA), np.sin(ALPHA)])
DELTA = 0.01 # параметр регуляризації r_j

```

```

N_PIX = 400                                # роздільність «екрана» N_PIX x N_PIX

P = resample(normalize(CONTOUR), M)
G = solve_gammas(P, V_INF, GAMMA0)

C, N = collocation(P)
U, V = velocity(C[:, 0], C[:, 1], P, G, V_INF, 0.0)
print(f"sum  $\Gamma$  = {G.sum():.6f} (має бути {GAMMA0}), max|V·n| у колокаціях = {np.abs(U * N[:, 0] + V * N[:, 1]).max():.1e}")

X, Y = grid(N_PIX)
U, V = velocity(X, Y, P, G, V_INF, DELTA)
fields = {
    "speed": np.hypot(U, V),
    "phi": phi_transformed(X, Y, P, G, V_INF, DELTA),
    "psi": psi_transformed(X, Y, P, G, V_INF, DELTA),
}

fig, ax = plt.subplots(2, 2, figsize=(10, 10))
Xs, Ys = grid(25)
Us, Vs = velocity(Xs, Ys, P, G, V_INF, DELTA)
ax[0, 0].quiver(Xs, Ys, Us, Vs)
ax[0, 0].set_title("1) векторне поле V")
for a, (key, title) in zip(ax.flat[1:], [("speed", "2) |V| = const"), ("phi", "3)  $\phi$  = const"), ("psi", "4)  $\psi$  = const")):
    F = fields[key]
    lv = np.linspace(*np.percentile(F, [1, 99]), 30)
    a.contour(X, Y, F, levels=lv, cmap="Blues" if key == "speed" else "RdBu_r")
    a.set_title(title)
for a in ax.flat:
    a.plot(P[:, 0], P[:, 1], "k-", lw=2)
    a.set_aspect("equal")
fig.savefig(f"lab1_G{GAMMA0:g}.png", dpi=150)

```

Джерела

1. Довгий С.О., Троценко Я.П., Черній Д.І. Технології чисельного моделювання. Лабораторний практикум: навч. посібник. — К., 2024. — 139 с. (розд. 7.1).
2. Презентація «Лабораторна №1 (розділ — Технологія)», слайди 1–17.
3. Довгий С.А., Лифанов И.К., Черний Д.И. Метод сингулярных интегральных уравнений и вычислительные технологии. — К.: Юстон, 2016. — 380 с.
4. Белоцерковский С.М., Ништ М.И. Отрывное и безотрывное обтекание тонких крыльев идеальной жидкостью. — М.: Наука, 1978. — 351 с.